

C# Interview Assessment Briefs

Exercise 1 — Bank Transfer Service

Estimated time: 45–60 minutes

Scenario: You are building a basic bank transfer service. Users should be able to transfer money between two accounts. This must work correctly even if several transfers happen at the same time.

Requirements:

Implement a `TransferAsync(fromId, toId, amount, requestId)` method.

The transfer should:

- Move money atomically from one account to another.
- Prevent double spending when multiple transfers happen at the same time.
- Reject invalid inputs (negative/zero amounts, same source and destination).
- Be idempotent — if the same `requestId` is provided twice, the second call should not re-execute the transfer.

Deliverables:

A working `TransferService` class that implements `ITransferService`.

Handle concurrency safely.

Handle invalid input gracefully with exceptions.

Write a short explanation of your approach and any assumptions you made.

Exercise 2 — Async HTTP Client Review

Estimated time: 30–45 minutes

Scenario: You join a team and are asked to review a small class that fetches user information from an API. The current implementation is unreliable and has several issues.

Requirements:

Refactor the provided class to:

- Be fully async/await (no blocking calls).
- Support cancellation with `CancellationToken`.
- Handle HTTP errors sensibly (e.g. 404 vs other errors).
- Be testable (e.g. by injecting `HttpClient`).

You may optionally add resilience features such as retries or timeouts.

Deliverables:

A refactored `UserClient` that avoids the issues.

A short description of the problems you identified.

(Optional) A simple unit test or example usage.

Exercise 3 — Product Search with LINQ and Paging

Estimated time: 40–50 minutes

Scenario: You are building a simple product catalog service. Users need to search products by name, SKU, or category, with sorting and paging support.

Requirements:

Implement a method: `Task<SearchAsync(ProductQuery query, CancellationToken ct = default);`

Search should:

- Filter by search term (Name or Sku, case-insensitive).
- Filter by category if provided.
- Sort by CreatedUtc, Price, or Name, in ascending or descending order.
- Return paged results (default 20 per page, maximum 100).
- Project to a lightweight ProductDto.

Deliverables:

A working LINQ query that returns Paged from DbSet.

Handle edge cases (page bounds, empty results, invalid sort input).

Write a short note explaining how your query avoids pulling too much data into memory.

■ Candidate Guidance

You may use C# 10+ features if you like.

Focus on clarity and correctness over micro-optimisation.

Add comments or short notes where you've made assumptions.

Partial solutions are fine — explain what you would do next if given more time.